

We will use only **5 algorithms**:

1. Logistic Regression
2. KNN
3. Decision Tree
4. Random Forest
5. Naive Bayes

The code follows your project flow exactly and avoids unnecessary complicated code.

Step 1: Load the dataset

Make sure `wine_quality.csv` is in the same folder as your Python file.

```
import pandas as pd

df = pd.read_csv("wine_quality.csv")

print(df.head())
print(df.shape)
print(df.columns)
```

What this does

- `read_csv()` loads the dataset.
- `head()` shows the first 5 rows.
- `shape` shows rows and columns.
- `columns` shows the column names

Step 2: Check the dataset

```
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
```

This checks:

- Missing values
- Duplicate rows

Step 3: Remove duplicates

```
df = df.drop_duplicates()

print("New shape:", df.shape)
```

Duplicate rows are removed.

Step 4: Handle missing values

```
df = df.fillna(  
    df.median(numeric_only=True)  
)  
  
print(df.isnull().sum())
```

Missing numerical values are replaced with the median.

Step 5: Feature Engineering

The original dataset has a `quality` score.

We convert it into two classes:

- 0 → Not Good Quality
- 1 → Good Quality

```
df["quality_class"] = df["quality"].apply(  
    lambda x: 1 if x >= 7 else 0  
)  
  
print(df[["quality", "quality_class"]].head())  
print(df["quality_class"].value_counts())
```

Step 6: EDA – Wine Quality

```
import matplotlib.pyplot as plt  
import seaborn as sns  
  
sns.countplot(  
    x="quality",  
    data=df  
)  
  
plt.title("Wine Quality Distribution")  
plt.show()
```

This shows how many wines belong to each quality score.

Step 7: EDA – Correlation Heatmap

```
import matplotlib.pyplot as plt  
import seaborn as sns  
  
sns.heatmap(  
    df.corr(numeric_only=True),  
    annot=True  
)  
  
plt.title("Correlation Heatmap")  
plt.show()
```

This helps us understand the relationship between features.

Step 8: Separate Features and Target

X contains the input features.

y contains the output.

```
X = df.drop(
    ["quality", "quality_class"],
    axis=1
)

y = df["quality_class"]

print("Features:")
print(X.columns)

print("\nTarget:")
print(y.head())
```

Step 9: Split the Dataset

We use:

- 80% → Training
- 20% → Testing

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Step 10: Feature Selection

We select the 8 most useful features.

```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_classif

select = SelectKBest(
    f_classif,
    k=8
)

X_train = select.fit_transform(
    X_train,
    y_train
)
```

```
X_test = select.transform(  
    X_test  
)  
  
print("Selected features:", X_train.shape[1])
```

Step 11: Feature Scaling

Scaling puts the features on a similar scale.

```
from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
  
X_train = scaler.fit_transform(  
    X_train  
)  
  
X_test = scaler.transform(  
    X_test  
)  
  
print("Scaling completed")
```

Step 12: Logistic Regression

```
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import accuracy_score  
  
modell = LogisticRegression(  
    max_iter=1000  
)  
  
modell.fit(  
    X_train,  
    y_train  
)  
  
p1 = modell.predict(  
    X_test  
)  
  
a1 = accuracy_score(  
    y_test,  
    p1  
)  
  
print("Logistic Regression:", a1 * 100, "%")
```

Step 13: KNN

```
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.metrics import accuracy_score
```

```
model2 = KNeighborsClassifier(  
    n_neighbors=5  
)  
  
model2.fit(  
    X_train,  
    y_train  
)  
  
p2 = model2.predict(  
    X_test  
)  
  
a2 = accuracy_score(  
    y_test,  
    p2  
)  
  
print("KNN:", a2 * 100, "%")
```

Step 14: Decision Tree

```
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import accuracy_score  
  
model3 = DecisionTreeClassifier(  
    random_state=42  
)  
  
model3.fit(  
    X_train,  
    y_train  
)  
  
p3 = model3.predict(  
    X_test  
)  
  
a3 = accuracy_score(  
    y_test,  
    p3  
)  
  
print("Decision Tree:", a3 * 100, "%")
```

Step 15: Random Forest

```
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import accuracy_score  
  
model4 = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42  
)  
  
model4.fit(  
    X_train,  
    y_train  
)  
  
p4 = model4.predict(  
    X_test  
)  
  
a4 = accuracy_score(  
    y_test,  
    p4  
)  
  
print("Random Forest:", a4 * 100, "%")
```

```

        X_train,
        y_train
    )

p4 = model4.predict(
    X_test
)

a4 = accuracy_score(
    y_test,
    p4
)

print("Random Forest:", a4 * 100, "%")

```

Step 16: Naive Bayes

```

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

model5 = GaussianNB()

model5.fit(
    X_train,
    y_train
)

p5 = model5.predict(
    X_test
)

a5 = accuracy_score(
    y_test,
    p5
)

print("Naive Bayes:", a5 * 100, "%")

```

Step 17: Compare All 5 Algorithms

```

import pandas as pd

results = pd.DataFrame({
    "Algorithm": [
        "Logistic Regression",
        "KNN",
        "Decision Tree",
        "Random Forest",
        "Naive Bayes"
    ],
    "Accuracy": [
        a1 * 100,
        a2 * 100,
        a3 * 100,
        a4 * 100,

```

```

        a5 * 100
    ]
})

```

```
print(results)
```

You will get something like:

	Algorithm	Accuracy
0	Logistic Regression	XX.XX
1	KNN	XX.XX
2	Decision Tree	XX.XX
3	Random Forest	XX.XX
4	Naive Bayes	XX.XX

The actual values will depend on your dataset.

Step 18: Find the Best Algorithm

```

best = results.loc[
    results["Accuracy"].idxmax()
]

print("Best Algorithm:", best["Algorithm"])
print("Best Accuracy:", best["Accuracy"], "%")
best = results.loc[
    results["Accuracy"].idxmax()
]

print("Best Algorithm:", best["Algorithm"])
print("Best Accuracy:", best["Accuracy"], "%")

models = {
    "Logistic Regression": model1,
    "KNN": model2,
    "Decision Tree": model3,
    "Random Forest": model4,
    "Naive Bayes": model5
}

best_model = models[
    best["Algorithm"]
]

prediction = best_model.predict(X_test)

print("Final Model:", best["Algorithm"])

```

The algorithm with the highest **test accuracy** is selected.

Step 19: Accuracy Comparison Graph

```

import matplotlib.pyplot as plt
import seaborn as sns

sns.barplot(
    x="Accuracy",

```

```
        y="Algorithm",
        data=results
    )

plt.title("Algorithm Accuracy Comparison")
plt.xlabel("Accuracy (%)")
plt.show()
```

Step 20: Select the Best Model

```
models = {
    "Logistic Regression": model1,
    "KNN": model2,
    "Decision Tree": model3,
    "Random Forest": model4,
    "Naive Bayes": model5
}

best_model = models[
    best["Algorithm"]
]

prediction = best_model.predict(
    X_test
)

print("Final Model:", best["Algorithm"])
```

Step 21: Calculate Final Performance

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

accuracy = accuracy_score(
    y_test,
    prediction
)

precision = precision_score(
    y_test,
    prediction
)

recall = recall_score(
    y_test,
    prediction
)

f1 = f1_score(
    y_test,
    prediction
)

print("Accuracy:", accuracy * 100, "%")
```



```
print("Precision:", precision * 100, "%")
print("Recall:", recall * 100, "%")
print("F1 Score:", f1 * 100, "%")
```

Step 22: Confusion Matrix

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(
    y_test,
    prediction
)

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```

Wine Quality Dataset



Data Collection



Data Understanding



Data Preprocessing



EDA



Feature Engineering



Feature Selection



Train-Test Split

80% / 20%



Feature Scaling



